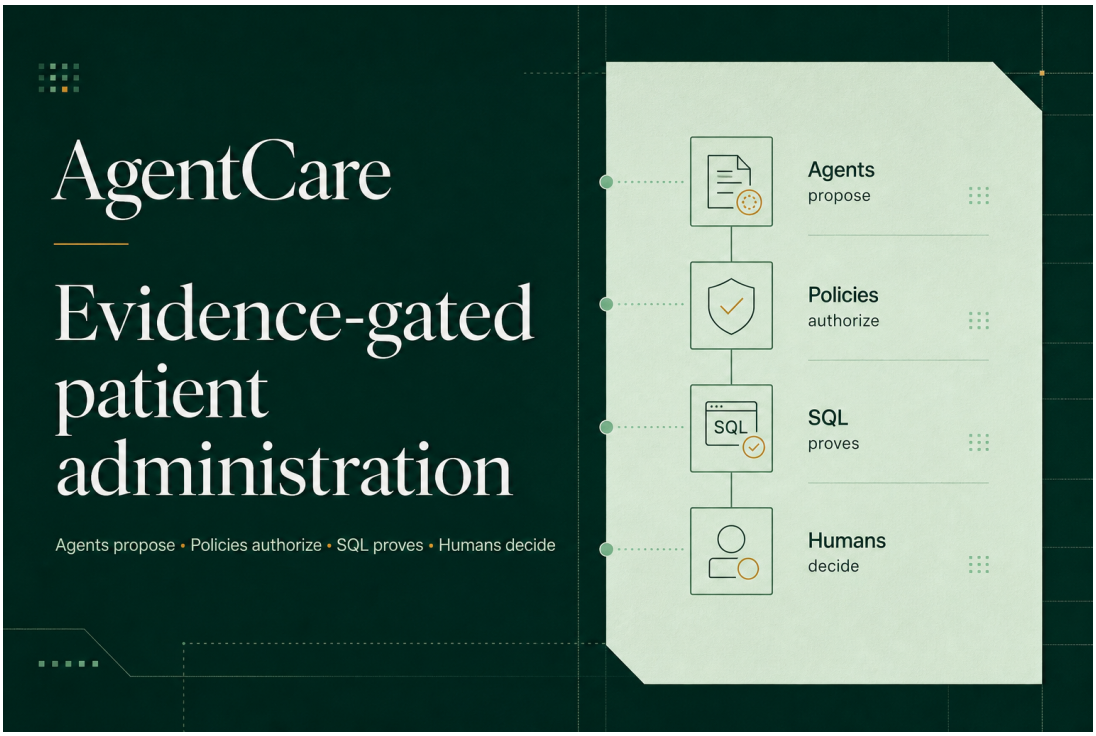


AgentCare Local Setup Guide

Clone, configure, run, verify, and test the complete evidence-gated patient administration platform.

Repository	github.com/chinmayk1613/AgentCare_AI_Patient_Administration_and_Care_Coordination
Branch	main
Recommended route	Docker Compose
Native stack	Python 3.11/3.12 + Node.js 22.13+ + pnpm 11.9
LLM	Optional developer-owned OpenAI key; safe fallback works without one

Agents propose. Policies authorize. Services execute. SQL proves. Humans decide exceptions.

Version 1.0 - July 28, 2026

1. What a new developer needs

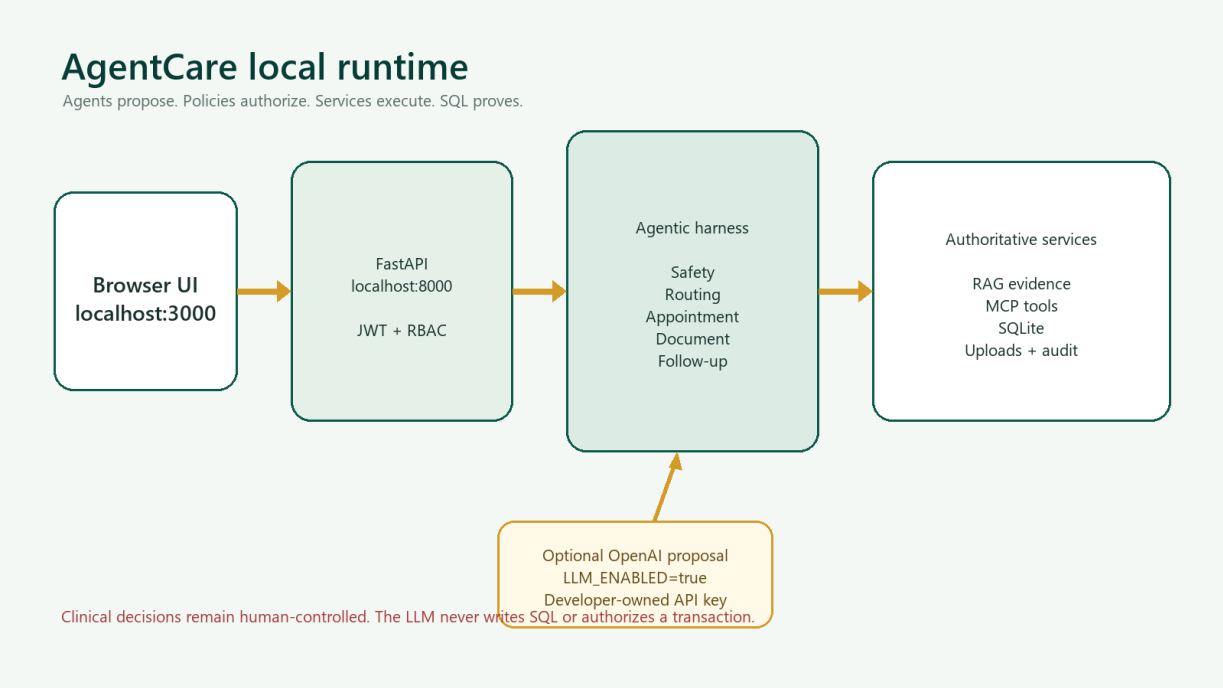
Choose the Docker route for the fastest complete startup. Choose the native route when editing or debugging the Python and TypeScript services separately.

Path	Required software	Best for
Docker - recommended	Git + Docker Desktop + Compose	Fast, isolated, complete startup
Native development	Git + Python 3.11/3.12 + Node 22.13+ + pnpm 11.9	Editing and debugging

- Optional OpenAI API key with active billing/quota for real LLM proposals.
- Editor such as VS Code.
- Synthetic document under 10 MB for upload tests.

No OpenAI key is required to exercise persistence, RAG retrieval, MCP tools, policy gates, booking, documents, human review, and audit evidence. The explicit safe-fallback path remains end-to-end.

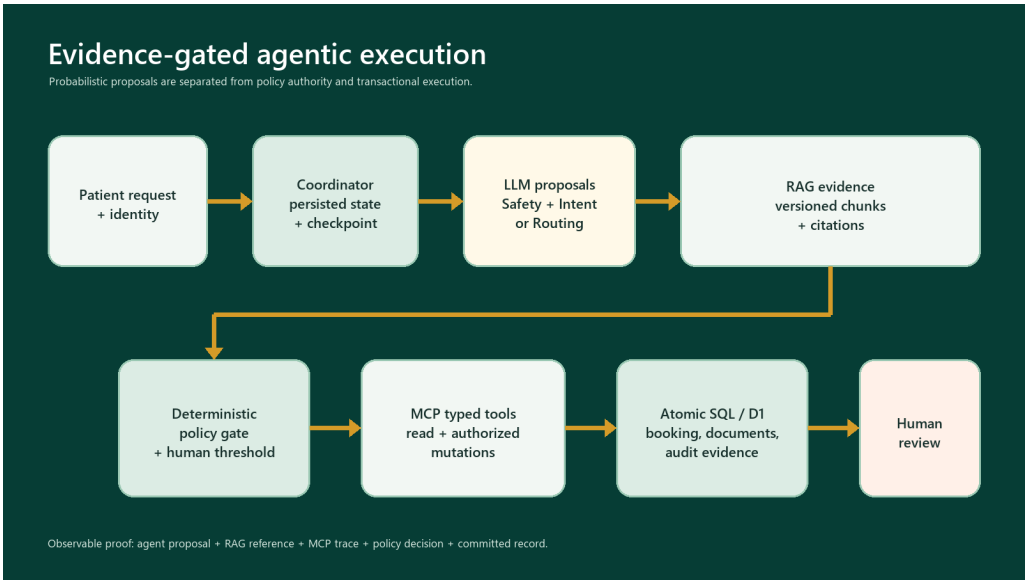
2. Local architecture



The browser calls FastAPI locally. The backend owns JWT/RBAC, agents, RAG, MCP-style tools, SQLite transactions, uploads, and audit events. Only bounded proposals may use OpenAI.

2A. Why this is an agentic AI project

AgentCare is a persisted, evidence-gated multi-agent workflow - not one chatbot prompt around an appointment form. The coordinator advances durable checkpoints; specialist agents propose or invoke bounded tools; policies and humans authorize; committed records establish truth.



Agent	Purpose	Boundary
Coordinator	State graph, hand-offs, truthful status	Workflow/checkpoints only
Safety	Emergency and clinical-boundary gate	Escalate + audit
Intent	Hosted book/reschedule/cancel proposal	Structured proposal only
Routing	Evidence-grounded department proposal	RAG, lookup, escalation
Appointment	Search/book/cancel/reschedule	Typed tools + atomic SQL
Document	Type, dedupe, security, requirements	No clinical interpretation
Follow-up	Reminders and administrative tasks	Committed records only

Python: backend/app/agents.py + orchestrator.py. Hosted: app/api/_agentic.ts + workflows/[workflowId]/advance/route.ts.

2B. Exact use of LLM, RAG, MCP, and fine-tuning

Capability	Exact implementation	Purpose
LLM - Python	openai-agents==0.2.10; Agent + Runner.run_sync; OPENAI_MODEL defaults to gpt-5-mini; max 3 turns	Structured Safety and Routing proposals
LLM - hosted	Direct fetch to OpenAI Responses API; strict JSON schema; deployment model gpt-5-mini	Structured Safety and Intent proposals
RAG	Versioned policy chunks; 128D local semantic hash; D1/SQL; cosine 45% + concepts 40% + lexical 15%	Ground routing, providers, document rules, and guardrails
MCP - Python	mcp==1.12.3 FastMCP: departments, approved policy, available slots	Typed read boundary over hospital services
MCP - hosted	JSON-RPC 2025-06-18: policy, RAG manifest, departments, slots, booking, cancel, reschedule, documents	Audited read/write tool calls after authorization
Fine-tuning	Optional OPENAI_FINE_TUNED_MODEL; only validated ft:* IDs; none currently claimed	Future task consistency - never policy or authorization

Where the LLM is used

When a patient submits a free-text request, the LLM helps the Safety Agent identify potential emergency or prohibited clinical language, the hosted Intent Agent classify booking, rescheduling, or cancellation, and the Routing Agent propose a hospital department using retrieved RAG evidence. The result is only a structured proposal; policy gates, human-review thresholds, MCP tools, and committed SQL records determine and prove the final action.

RAG behavior

- Catalog, terminology, document rules, providers, and guardrails are parsed and chunked.
- Evidence references include policy key, version, and chunk number.
- Live slots are excluded from RAG and queried transactionally through MCP.

Fine-tuning truth

The current project uses a base model or deterministic fallback. A fine-tuned model is not required and is not falsely claimed. Policies, slots, authorization, and safety thresholds stay in RAG, MCP/SQL, and deterministic code.

Reviewer-visible proof

- agent_proposals: agent, decision, confidence, model, execution mode.
- RAG: chunk ID, policy version, score, excerpt, embedding model.
- MCP: server, transport, tool, input/output, status, timestamp.
- Audit + SQL/D1: checkpoints, escalations, human decisions, committed records.

3. Clone and inspect

Windows PowerShell, macOS, or Linux:

```
git clone https://github.com/chinmayk1613/AgentCare_AI_Patient_Administration_and_Care_Coordination.git
cd AgentCare_AI_Patient_Administration_and_Care_Coordination
git switch main
git status
```

Expected: branch main, clean working tree, and no real .env file or credential.

4. Recommended route - Docker Compose

Step 1 - Verify Docker Desktop is running

```
docker --version
docker compose version
```

Step 2 - Create the ignored local environment

```
Copy-Item .env.example .env    # Windows PowerShell
cp .env.example .env           # macOS/Linux
```

```
JWT_SECRET=replace-with-a-long-random-local-secret
LLM_ENABLED=false
OPENAI_API_KEY=
OPENAI_MODEL=gpt-5-mini
```

Step 3 - Build and start

```
docker compose up --build
```

- API: http://localhost:8000
- Web: http://localhost:3000
- API docs: http://localhost:8000/docs
- Persistent local volume: agentcare_data

Step 4 - Verify health

```
curl http://localhost:8000/health
# {"status":"ok","llm_mode":"safe-fallback","write_tools":true}
```

4. Docker lifecycle and reset

Stop while preserving the synthetic database and uploads:

```
docker compose down
```

Restart without rebuilding:

```
docker compose up
```

Delete only the local Docker volume and reseed:

```
docker compose down -v
docker compose up --build
```

The -v command destroys the local AgentCare volume. Do not use it against important data.

5. Native backend setup

Step 1 - Verify runtimes

```
git --version
python --version
node --version
pnpm --version
```

Expected: Python 3.11 or 3.12, Node.js 22.13+, and pnpm 11.9.

Step 2 - Create a virtual environment

```
# Windows PowerShell
python -m venv .venv
./.venv/Scripts/Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

# macOS/Linux
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Step 3 - Copy backend environment template

```
Copy-Item backend\.env.example backend\.env    # Windows
cp backend/.env.example backend/.env             # macOS/Linux
```

5. Native services

Recommended backend/.env values:

```
APP_ENV=development
DATABASE_URL=sqlite:///./data/agentcare.db
JWT_SECRET=replace-with-a-long-random-local-secret
OPENAI_API_KEY=
OPENAI_MODEL=gpt-5-mini
LLM_ENABLED=false
UPLOAD_DIR=./data/uploads
MAX_UPLOAD_MB=10
WRITE_TOOLS_ENABLED=true
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:5173
```

Step 4 - Seed and start FastAPI

```
cd backend
python -m app.seed
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Expected: backend/data/agentcare.db, synthetic hospital data, and HTTP 200 from /health.

Step 5 - Start the web interface in a second terminal

```
pnpm install --frozen-lockfile
pnpm run dev
```

Open http://localhost:3000. Localhost defaults to the API at http://127.0.0.1:8000.

6. Optional OpenAI LLM

```
LLM_ENABLED=true
OPENAI_API_KEY=your-own-key-here
OPENAI_MODEL=gpt-5-mini
```

Restart FastAPI. The /health response should show llm_mode: enabled. Each developer must use an authorized key with active quota.

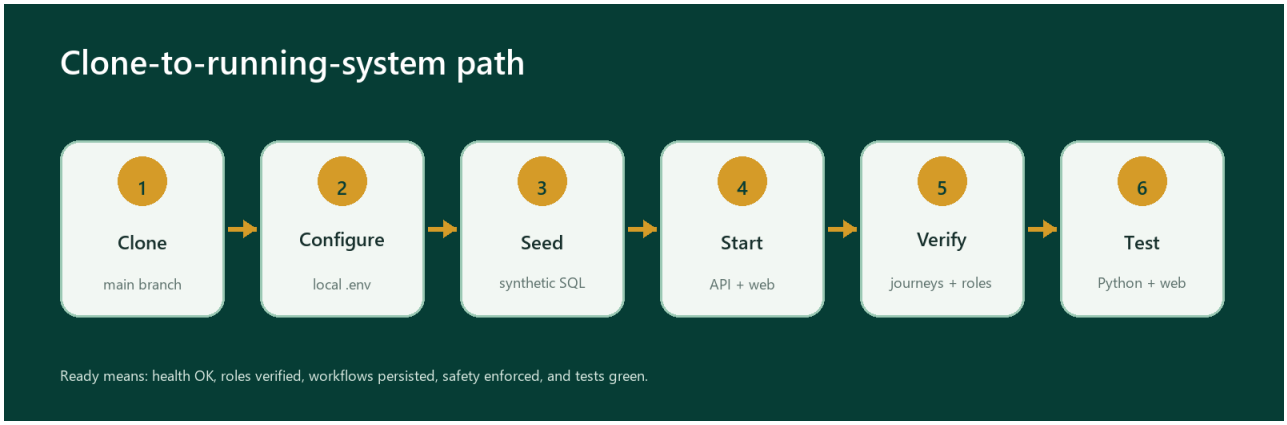
Never commit or share an API key. A provider error returns the workflow to the safe-fallback path. The model never authorizes SQL writes or clinical decisions.

7. Synthetic accounts

Role	Email	Password
Patient - Chinmay Kashikar	chinmay.kashikar@agentcare.demo	Patient123!
Patient - Mayuresh Kashikar	mayuresh.kashikar@agentcare.demo	Patient123!
Staff - Dr Vikas Jha	vikas.jha@agentcare.demo	Reviewer123!
Staff - Dr Arunima Gosavi	arunima.gosavi@agentcare.demo	Reviewer123!

Demonstration only - not for clinical use. Use synthetic data only; never enter or upload real PHI. Public-demo uploads require confirmation, are rate-limited, and accept PDF, PNG, JPEG, or TXT up to 10 MB.

8. End-to-end verification



A. Normal journey - Sign in as the patient and submit:

```
I need a cardiology follow-up next week. I also want to attach my previous ECG.
```

- Confirm stepwise safety, intent, RAG/MCP routing, live slot selection, appointment commit, document validation, reminder, case number, and audit evidence.

B. Human approval - Submit:

```
My legs are painful. I need to consult a doctor and submit my MRI report.
```

- Confirm staff review opens case details, approval resumes the same workflow, and live availability follows.

C. Document mismatch - Require MRI, upload a synthetic ECG, and verify MRI stays missing with a warning.

D. Safety boundary - Ask for diagnosis/prescription and verify escalation without clinical advice.

9. Automated tests

```
cd backend
python -m pytest
cd ..

pnpm run lint
pnpm run build
node --test tests/rendered-html.test.mjs
```

- Python source compiles and backend tests pass.
- Persistence, idempotency, safety, and RBAC behavior pass.
- TypeScript lint and Cloudflare Worker build pass.
- Rendered interface test confirms the real AgentCare product and hosted API.

10. Update an existing clone

```
git status
git pull --ff-only origin main
python -m pip install -r requirements.txt
pnpm install --frozen-lockfile
```

For Docker, use docker compose up --build after pulling changes.

11. Security and data rules

- Never commit .env, credentials, patient documents, local databases, uploads, or logs.
- Use synthetic or properly anonymized data only.
- Rotate a key immediately if it appears in chat, logs, screenshots, or Git history.
- Replace demo JWT login with approved OIDC/OAuth 2.1 before non-demo use.
- The system is administrative; it must not diagnose, prescribe, dose, or interpret findings.

```
git status
git diff --check
git grep -n "OPENAI_API_KEY="
```

12. Troubleshooting

Symptom	Likely check	Resolution
Backend offline	/health on port 8000	Start API; verify API base URL
Port conflict	Local process/container	Stop conflict or update URLs/origins together
Login fails	Seeded database	Run app.seed or recreate local Docker volume
LLM stays fallback	Flag, key, quota, restart	Set both values; use active quota; restart
Upload fails	Size/type/directory	Use synthetic file under 10 MB; check write access
CORS error	Allowed origins/API URL	Use localhost origins and matching API base
No slot	Seed/bookings	Use another specialty/date or fresh local database
Build failure	Node/pnpm/lockfile	Use Node 22.13+ and frozen install
CI syntax mismatch	Python version	Use Python 3.11/3.12; run tests

13. Completion checklist

Check	Pass condition
Repository	main selected; clean; no secrets
Configuration	Ignored local .env files created
Database	Synthetic seed completed
API	/health returns status: ok
Web	Port 3000 opens and patient login works
Roles	Staff/reviewer case access works
Workflow	Normal journey reaches slot confirmation
Human review	Paused case resumes after approval
Documents	Mismatch remains missing and warns
Safety	Clinical request escalates without advice
Tests	Backend, lint, build, and UI tests pass

When every row passes, the clone is ready for development or evaluation.

Repository: https://github.com/chinmayk1613/AgentCare_AI_Patient_Administration_and_Care_Coordination